Partie N°5 – Create Ract App

1. Découvert de Create React App (CRA)

Comprendre l'importance des outils automatisés ?

Nous avons appris à utiliser les liens **CDN** de **React**, **ReactDOM**, et à paramétrer **Babel** pour créer une **app React**. Mais cette technique n'est quasiment pas utilisée dans la vie de tous les jours d'un développeur.

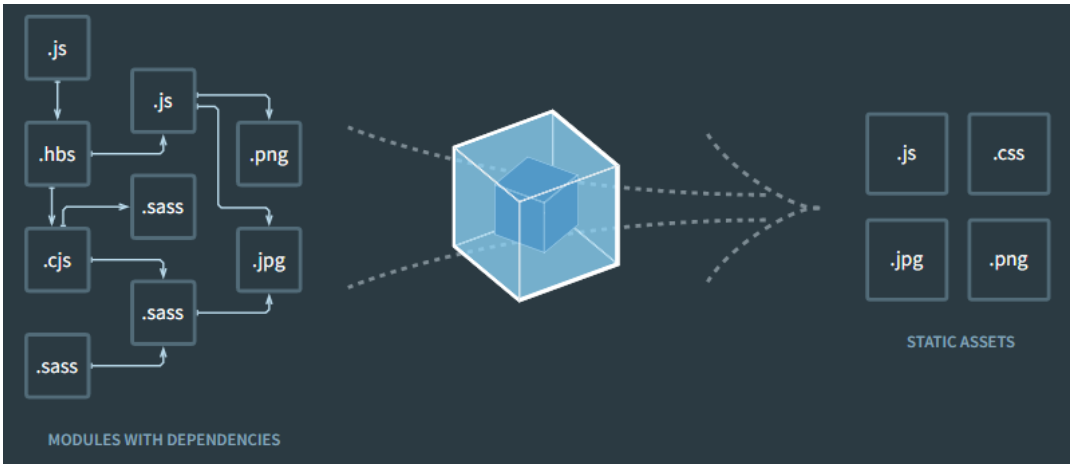
À la place, les développeuses utilisent des outils automatisés pour créer une base de code, qui dispose des outils essentiels déjà préconfigurés. Ces outils permettent de :

- Gérer les différentes dépendances (bibliothèques) utilisées par notre app.
- Optimiser le chargement de notre code dans les navigateurs.
- Importer du CSS et des images ;
- Gérer les différentes versions de JavaScript ;
- Faciliter l'expérience de développement, en rechargeant la page lorsque le code est modifié.

Découvrir CRA

Create React App https://create-react-app.dev/ est un outil crée par l'équipe de Facebook, qui permet de générer un squelette de code pour votre application. Il embarque un certain nombre d'outils préconfigurés, tels que **Webpack**, **Babel**, et **ESLint**, afin de vous garantir la meilleure expérience de développement possible :

- Le **Webpack** est ce qu'on appelle un **module bundler** est un outil principalement utilisé en **front-end**, et qui va permettre de rassembler tous les fichiers de votre code source et les rassembler en un seul fichier.

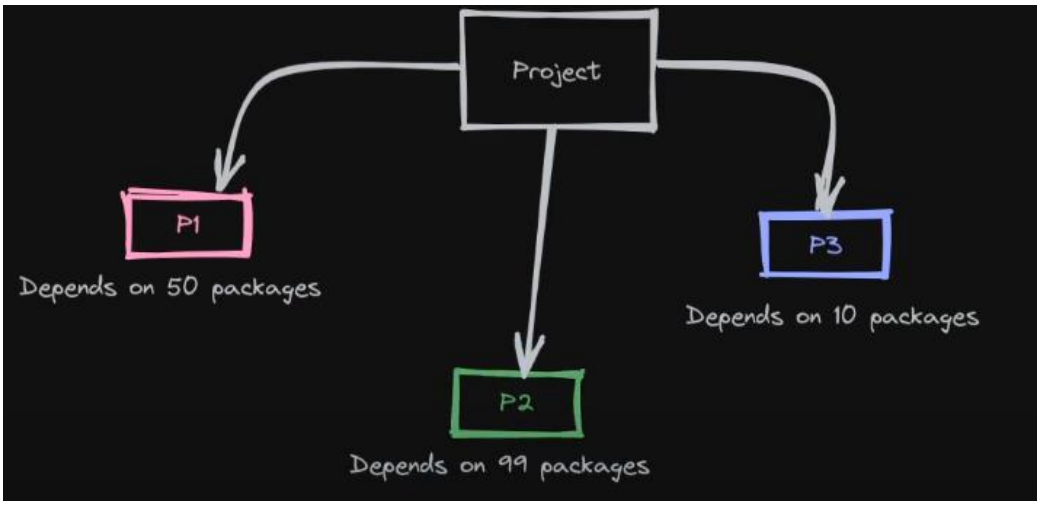


- **Babel** : Babel est **toolchain** principalement utilisée pour convertir le code **ECMAScript 2015+** en une version compatible de JavaScript dans les navigateurs ou environnements actuels et anciens.
- **ESLint** est un outil d'analyse de code statique permettant d'identifier les modèles problématiques trouvés dans le code JavaScript.

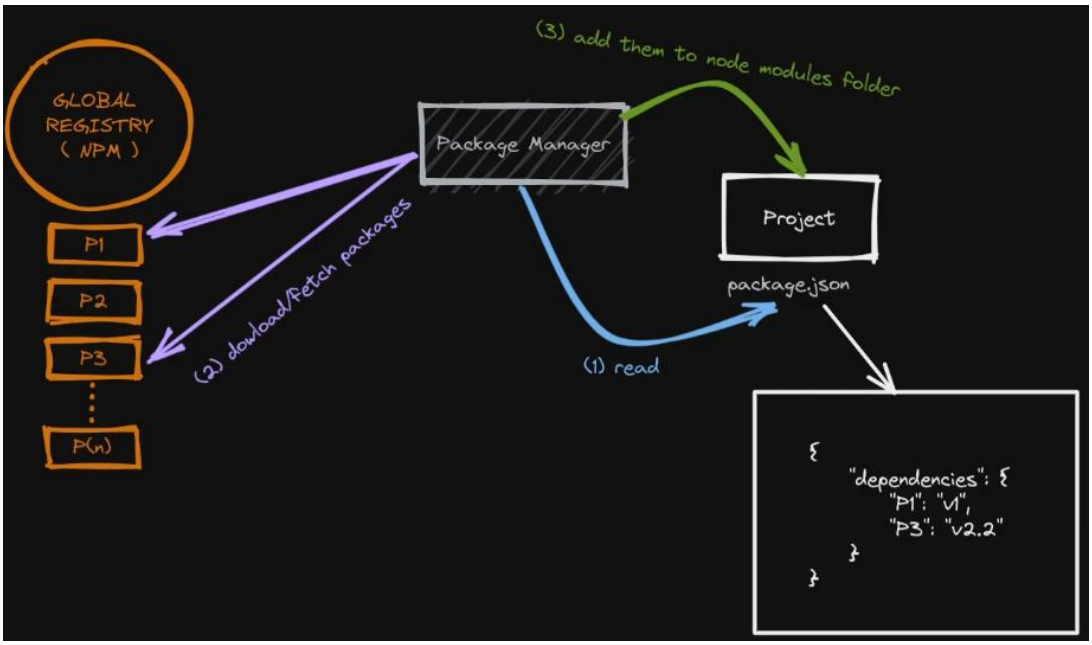
C'est quoi un package manager ?

Un package est une partie de code réutilisable téléchargé à partir d'un registre global (**Global registry**) et sauvegarder vers l'environnement local de développement. Un package peut avoir des dépendances vers d'autres packages. Par exemple si on 'a besoin d'un site e-commerce d'une fonctionnalité qui va faire la conversion en plusieurs type de devises on peut télécharger un package qui offre cette fonctionnalité. Ce téléchargement se fait à travers **un package manager**.

Un package manager (par exemple **npm node package manager**) est une partie d'un logiciel qui permet de télécharger des packages et aussi de gérer les dépendances entre les packages pour qu'ils fonctionnent correctement :



Le package manager permet aussi de mettre ajours un package et aussi de le supprimer. La figure suivante explique le fonctionnement d'un package manager :

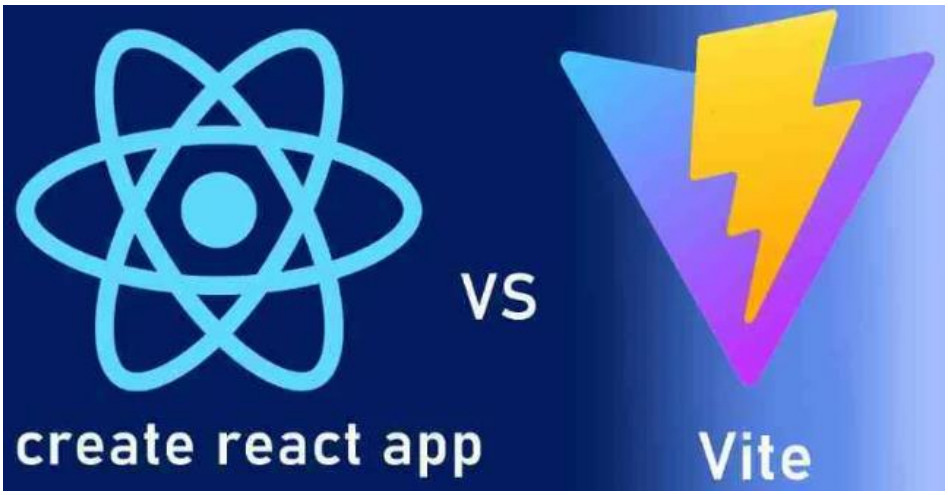


Pour utiliser un package manager, chaque projet doit contenir un fichier **package.json** ,ce fichier contient tous les packages nécessaire au fonctionnement du projet. Le travail d'un package manager se fait en 3 étapes :

1. La package manager lit le fichier **package.json** pour voir les dépendances et les packages qu'il doit installer et dans notre cas il s'agit des packages **P1** et **P3**.
2. Le **Global Registry** est un emplacement utilisé par le **npm** pour télécharger les packages et dans notre cas il va télécharger les deux packages **P1** et **P2**.
3. Le **npm** va sauvegarder les deux packages **P1** et **P2** téléchargés dans le dossier **node modules**.

Remarque : On peut utiliser comme package manager : **npm**, **yarn** ou le **pnpm**.

Create React App vs vite



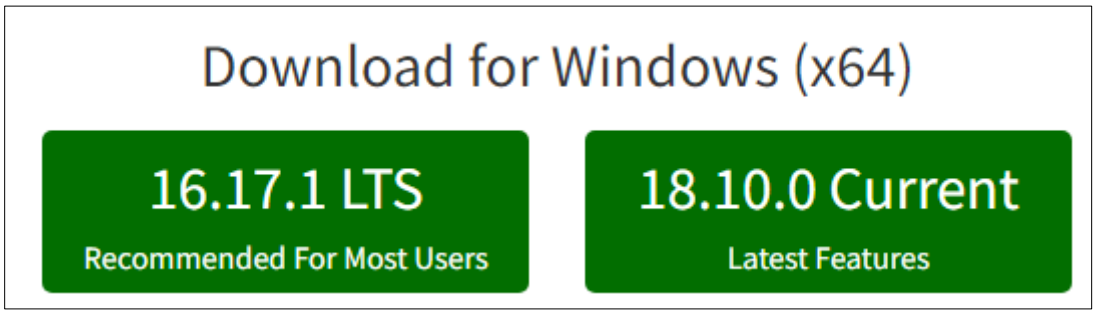
Il y'a deux façon pour créer un projet **React** :

- **Create React App** (**CRA**) est depuis longtemps l'outil incontournable pour de nombreux développeurs de tous niveaux lorsqu'il s'agit de créer une application React (débutants, intermédiaires et même experts). Il présente cependant quelques inconvénients importants, qui sont la vitesse et les performances**.**
- **Vite** est un outil de construction frontal de nouvelle génération créé par **Evan You** qui est le créateur de **Vue.js**. Il fonctionne très rapidement et possède de nombreuses fonctionnalités intéressantes fournies par CRA.

Les liens ci-dessous présentent la documentation de **Create React App** et **Vite** :
https://create-react-app.dev/docs/getting-started
https://vitejs.dev/guide/

Installer et lancer CRA

Pour créer une **app React**, nous allons besoin d'installer **Node.js**.
Le **Node.js** est un environnement d'exécution JavaScript open source et multiplateforme que vous pouvez télécharger à partir du site https://nodejs.org/en/ et l'installer sur votre machine :



Le **Noje.js** dispose du gestionnaire de paquet **npm** (node package manager) qu'on va utiliser pour créer une **app React** ou bien pour installer les autres package manager. Voici les différentes possibilités pour créer une **app React** :

- Pour créer un projet React avec npx :
  ```
  npx create-react-app my-app
  ```
- Pour créer un projet React avec npm :
  ```
  npm init react-app my-app
  npm create vite my-app
  ```

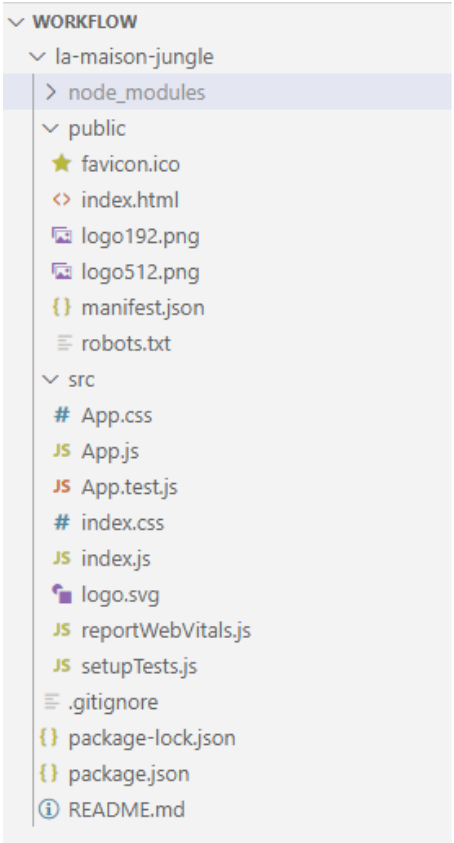
Pour commencer, créer un dossier **workflow** et placez-vous dans ce dossier où qui va contenir votre projet.
Pour initialiser votre projet, nous allons faire :

```
npx create-react-app la-maison-jungle
```

Maintenant que votre projet est initialisé, On rentre dans le projet avec :

```
cd la-maison-jungle
```

À partir de là, vous pouvez ouvrir le projet dans **vscode** dont voici son architecture :



- Le dossier **node_modules** : contient toutes les librairies JavaScript nécessaires à l'application.
- Le dossier **public** contient le fichier **index.html** qui est notre point départ avec la **div** qui a pour class **root** qui permet de signifier a **React** ou est ce qu'il doit venir s'ancrer.
- Le fichier **.gitignore** : précise quels fichiers qui ne doivent pas être pusher dans git sur le repo distant notamment le dossier **node_modules**.
- **Package.json** : contient beaucoup d'information sur l'application **React**, notamment les dépendances et les scripts
    - **start** : que vous pouvez exécuter **npm start** qui permet de lancer l'application.
    - **build** : qui permet de générer un dossier optimisé pour pouvoir déployer l'application en production.
    - **test :** pour faire des tests.
    - **eject :** reconfigurer l'application à la main.
- Le fichier **README.md** permet de documenter votre application.
- Le dossier **src** on y retrouve :
    - Le fichier **index.js** : qui est le fichier qui charge notre premier composant le composant **App** qui est ensuite greffer a la **div** avec la class **root** qui se trouve dans le fichier **index.html** au niveau du dossier **public**.
    - Le fichier **App.js** : que vous pouvez modifier pour voir le résultat.

Le fichier public/index.html :

Ce fichier ne contient pas grand-chose, c'est là qu'est défini la structure de base d'une page HTML, plus important il y est défini l'id **root** :

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />

    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />

    <title>React App</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>

  </body>
</html>
```

Le fichier src/index.js :

Ce fichier va permettre de récupérer nos composants et les afficher dans la **div#root** de **public/index.html** :

la-maison-jungle > src > JS index.js > ...

You, 1 second ago | 1 author (You)

```
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import './index.css';
4  import App from './App';
5  import reportWebVitals from './reportWebVitals';
6
7  const root = ReactDOM.createRoot(document.getElementById('root'));
8  root.render(
9    <React.StrictMode>
10     <App />
11   </React.StrictMode>
12 );
13
14 reportWebVitals();
```

- **La ligne 1** : importe la bibliothèque **React** elle-même. Comme **React** transforme l'instruction JSX que nous écrivons en **React.createElement()**, tous les composants **React** doivent importer le module **React**. Si vous ignorez cette étape, votre application produira une erreur.
- **La ligne 2** : importe Le module **ReactDOM** qui fournit des méthodes spécifiques au DOM que vous pouvez utiliser au niveau racine de votre application.
- **La ligne 3** : importe la feuille de style **src/index.css**.
- **La ligne 4** : importe le fichier **src/App.js** qui définit une fonction nommée **App**.
- **La ligne 8** : affiche l'élément **React <App />** au sein du nœud **DOM root** et renvoie une référence sur le composant**.**
- **Ligne 5** et **14** : **Web Vitals** sont un ensemble de mesures utiles qui visent à capturer l'expérience utilisateur d'une page Web. Avec la fonction **reportWebVitals**, vous pouvez envoyer n'importe résultats à un service d'analyse d'audience comme **Google Analytics**.

Le fichier src/App.js :

la-maison-jungle > src > JS App.js > ...

You, 1 second ago | 1 author (You)

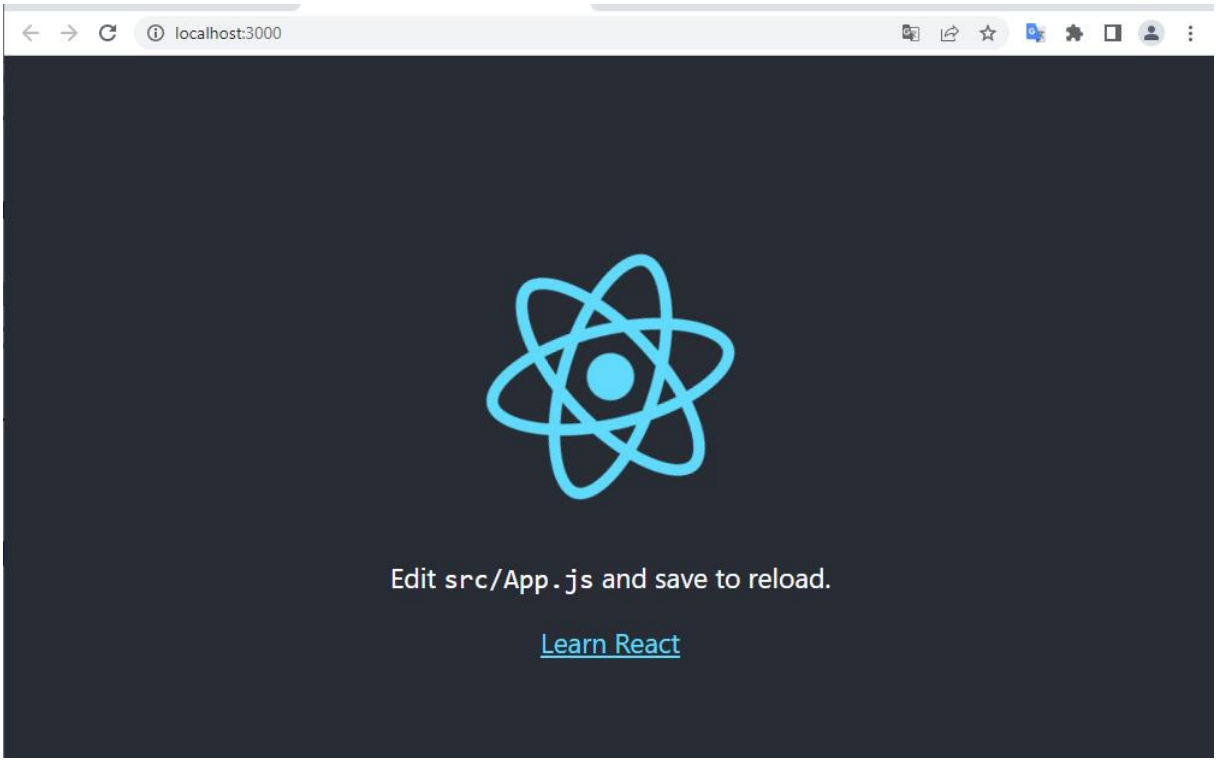
```
1  import logo from './logo.svg';
2  import './App.css';
3
4  function App() {
5    return (
6      <div className="App">
7        <header className="App-header">
8          <img src={logo} className="App-logo" alt="logo" />
9          <p>
10           Edit <code>src/App.js</code> and save to reload.
11         </p>
12         <a
13           className="App-link"
14           href="https://reactjs.org"
15           target="_blank"
16           rel="noopener noreferrer"
17         >
18           Learn React
19         </a>
20       </header>
21     </div>
22   );
23 }
24
25 export default App;
```

La fonction **App** renvoie une expression **JSX.** Cette expression définit ce que votre navigateur restitue finalement au **DOM**.

Tout en bas du fichier **App.js**, l'instruction export default App rend notre composant App disponible pour les autres modules.

Pour lancer votre application taper la commande :

```
npm start
```



Organiser votre code

Il existe plusieurs manières d'organiser son code, et il est important de réfléchir à comment l'organiser. Ici, nous allons séparer les fichiers selon leur type : composants/style/images, etc.

On va commencer par créer un dossier **/components** dans **/src**, où nous mettrons tous nos composants. On y glisse **App.js** et on en profite pour changer le chemin d'import dans **index.js** :

```
import App from './components/App'
```

Toujours dans le dossier **src**, supprimer les autres fichiers et garder seulement **index.js** et **index.css**.

Maintenant, créons notre component **Banner** sous forme de fonction dans un fichier JavaScript **Banner.js** dans le dossier **/components** :

```
function Banner() {
    return <h1>La maison jungle</h1>;
}

export default Banner;
```

On peut maintenant adapter le code d'**App.js** en supprimant le code de base, et y importer notre **Banner** de la façon suivante :

```
import Banner from "./Banner";
function App() {
  return (
    <Banner/>
  );
}
export default App;
```

Remarque : c'est **Webpack** qui nous permet d'importer notre composant aussi facilement, avec **import.**

Annexe N°1 :

Ci-dessous un fichier **PDF** en annexe sur les étapes de la création d'un composant.
PDF : P05_Annexe1.pdf

Atelier N°1 :

Vous trouverez ci-dessous le lien pour commencer l'atelier
Lien : https://codepen.io/soufiane-aittaleb/pen/QWrzEjX
Voilà ce qui est attendu de vous :

- Créer le component **Cart** dans un fichier **Cart.js** dans le dossier **/components**.
- Copier le code du component **Cart** à partir du lien pour commencer l'atelier.
- Transformer le fichier **Cart.js** en module : utiliser ici **export default**
- Importer le module **Cart** et invoquer le dans **App.js**.

Solution : https://github.com/SOUFIANE-AIT-TALEB/react2022/tree/P05Atelier1-Solution

Resumé

- Les développeurs utilisent des outils automatisés pour faciliter leur expérience de développement.
- **Create React App** (CRA) est la boîte à outils créée par Facebook, qui reste encore la référence pour initier un projet **React**.
- Un projet initialisé avec CRA possède toujours :
  - Un fichier **index.html** qui est le template où vivra notre **app React**.
  - Un fichier **package.json** qui liste les dépendances et les scripts.
  - Un fichier **index.js** dans lequel notre **app React** est initialisée, et greffée au HTML.
- CRA s'exécute avec l'aide d'un gestionnaire de paquet **npm**.
- **Webpack** permet d'importer simplement les fichiers entre eux.

Activité N°3 :

Vous trouverez ci-joint le **PDF ActivitéN°3.pdf** :
PDF : Activite3.pdf